## 3. Entregable 03 · Service Layer (Application Services)

**Principio:** un método por caso de uso. Los servicios **solo coordinan**: cargan Aggregates desde repositorios, los invocan, guardan y publican Domain Events. **No contienen reglas de negocio** (esas viven en el dominio: ADR-001 y ADR-005). Cada método se ejecuta en **una transacción** y es el punto de entrada único para el PdV Windows (ADR-007).

### 3.1 `VentaAppService` (contexto Ventas y Caja)

| Método (caso de uso) | US | Actor | Qué orquesta (sin lógica) |
|---|---|---|---|
| ConsultarDisponibilidad (codigoONombre) | US01 | Vendedor | ProductoRepository + consulta de solo lectura a StockSucursal de las 5 sucursales; devuelve disponible y apartado por separado |
| IniciarVenta(vendedorId, sucursalId, clienteDoc?) | US02 | Vendedor | Crea Venta en estado En proceso |
| AgregarProducto(ventaId, sku, cantidad) | US02, US05 | Vendedor | Carga Venta, Producto, StockSucursal; el dominio rechaza si el stock está RESERVADO (alerta "Producto reservado para [Cliente]") |
| RegistrarSerial(ventaId, sku, valorSerie) | US03 | Vendedor | Carga NumeroDeSerie; el dominio valida que no esté vendido; queda bloqueado para otra venta |
| PagarEnEfectivo(ventaId, montoRecibido) | US13 | Cajero | Delega en Venta.registrarPago; registra en talonario de caja |
| PagarConTarjetaOTransferencia(ventaId, metodo, ultimos4, banco, referencia) | US14 | Cajero | Igual, con DatosPagoElectronico |

| Método (caso de uso) | US | Actor | Qué orquesta (sin lógica) |
|---|---|---|---|
| ConfirmarVenta(ventaId) | US02 | Cajero | Venta.confirmar() → VentaRegistrada; StockSucursal.descontar; NumeroDeSerie.vender; GeneradorFolio; ComprobanteRepository.guardar |
| CancelarVenta(ventaId, motivo) | BPMN ("Cancelar transacción") | Cajero | Libera seriales; registra auditoría |
| ImprimirComprobante(folio) | US12 | Cajero | Comprobante.marcarImpreso() y puerto de impresión del PdV |

### 3.2 ApartadoAppService (contexto Apartados)

| Método | US | Actor | Qué orquesta |
|---|---|---|---|
| ConsultarApartadosDeCliente(documento) | US15 | Vendedor | Lectura; marca los próximos a vencer |
| CrearApartado(clienteDoc, sku, cantidad, montoAnticipo, metodoPago) | US04 | Vendedor / Cajero | Apartado.crear (el dominio valida ≥ 25% y fija vencimiento = hoy + 14 días); StockSucursal.reservar; Comprobante tipo Apartado |
| CompletarApartado(apartadoId, pago) | US06 | Cliente / Cajero | Apartado.completar → ApartadoCompletado; crea Venta formal con nuevo folio; StockSucursal pasa de reservado a descontado; descuenta serial si aplica |
| CancelarApartadosVencidos(fechaCorte) | US07 | Sistema (fin de día) | Busca fechaVencimiento < hoy y estado Activo; por cada uno: Apartado.cancelarPorVencimiento, StockSucursal.liberar, |

| Método | US | Actor | Qué orquesta |
|---|---|---|---|
| | | | notificación al cliente, auditoría. Idempotente (ADR-004) |

### 3.3 `DevolucionAppService` (contexto Devoluciones)

| Método | US | Actor | Qué orquesta |
|---|---|---|---|
| SolicitarDevolucion(folio?, sku, serial?, motivo) | US08 | Cajero / Vendedor | Carga Venta; PoliticaDevolucionService.evaluar (plazo 30 días, comprobante, > $500); crea Devolucion |
| RegistrarInspeccion(devolucionId, estado) | US08, US09 | Vendedor / Almacén | Devolucion.registrarInspeccion (Aceptable / Dañado) |
| AutorizarExcepcion(devolucionId, supervisorId, aprobada, motivo) | US09 | Supervisor | Devolucion.aplicarAprobacion; si se rechaza guarda motivo |
| RechazarDevolucion(devolucionId, motivo) | US09 | Supervisor | Registra motivo y muestra "No aplica porque [motivo]" |
| ResolverDevolucion(devolucionId, tipo, nuevoSku?) | US08 | Cajero | Reembolso por método original, o Cambio con diferencia (pagar, crédito, directo); StockSucursal.incrementar; NumeroDeSerie.registrarDevolucion; Comprobante tipo Devolución |

### 3.4 `InventarioAppService` (contexto Inventario y Seriales)

| Método | US | Actor | Qué orquesta |
|---|---|---|---|
| ReportarFaltante(sucursalId, sku, cantidadFisica, causa) | US10 | Almacén | Crea ReporteFaltante; ConciliadorInventario calcula diferencia; |

| Método | US | Actor | Qué orquesta |
|---|---|---|---|
| | | | genera tarea de investigación |
| AprobarAjuste(reporteId, supervisorId) | US10, regla 11 | Supervisor | Aplica el ajuste a StockSucursal |
| BuscarHistorialSerial(valorSerie) | US11 | Almacén / Supervisor | Lectura del historial de NumeroDeSerie (venta, cliente, sucursal, vendedor, devolución) |
| IniciarTransferencia(sku, cantidad, origenId, destinoId) | US16 | Supervisor | Crea TransferenciaInventario; StockSucursal.bloquearEnTransito en origen |
| RecibirTransferencia(transferenciaId) | US16 | Almacén | Descuenta origen, suma destino |
| ConsultarStockPorSucursal() | US17 | Supervisor | Lectura: Producto \| Suc1…Suc5 \| Total, separando disponible y apartado |

### 3.5 Servicios transversales

AuditoriaEventHandler (suscrito a todos los Domain Events, escribe RegistroAuditoria con timestamp, actor y justificación — regla 9) y NotificacionService (aviso al cliente por apartado cancelado, US07).

**Trazabilidad US → servicio:** US01→Venta, US02→Venta, US03→Venta, US04→Apartado, US05→regla en dominio (invocada desde AgregarProducto), US06→Apartado, US07→Apartado, US08→Devolución, US09→Devolución, US10→Inventario, US11→Inventario, US12→Venta, US13→Venta, US14→Venta, US15→Apartado, US16→Inventario, US17→Inventario. Las 17 historias quedan cubiertas.

## Revisión Obligatoria de la Service Layer

Para asegurar que la capa de aplicación mantenga una separación de responsabilidades estricta y no absorba la lógica de negocio, se sometió el diseño completo de los servicios (*VentaAppService*, *ApartadoAppService*, *DevolucionAppService* e *InventarioAppService*) a la siguiente revisión obligatoria de diseño:

- ¿El nombre del método expresa una acción del negocio?

  *Sí.* Cada método utiliza verbos y términos de dominio alineados con las historias de usuario y el lenguaje ubicuo de TecnoStock (por ejemplo, IniciarVenta, CrearApartado, SolicitarDevolucion, ReportarFaltante), permitiendo que cualquier analista o stakeholder identifique de inmediato la intención operativa sin necesidad de leer código técnico.

- ¿El método coordina o contiene decisiones que deberían estar en el dominio?

  *Estrictamente coordina.* Los *Application Services* actúan únicamente como fachadas de orquestación transaccional: reciben las solicitudes, cargan los agregados a través de los repositorios, invocan los métodos de comportamiento internos y guardan los cambios. Ninguna regla crítica (como validar el 25% de anticipo, el plazo de 30 días o la verificación de stock reservado) se calcula dentro del servicio; residen de forma pura dentro de las entidades y agregados del dominio.

- ¿El servicio depende de interfaces o de detalles concretos de base de datos?

  *Depende exclusivamente de abstracciones.* Los servicios interactúan con las capas inferiores mediante interfaces de repositorio (*Repository*) y contratos de dominio, aislando por completo los detalles tecnológicos de persistencia (SQL, ORM) o de la interfaz de usuario del punto de venta Windows.

- ¿La operación tiene un inicio y un final transaccional comprensible?

  *Sí.* Cada método de la *Service Layer* delimita una unidad de trabajo transaccional clara (*Unit of Work*). Si ocurre un fallo durante el registro de pagos, la asignación de seriales o el descuento de existencias, se ejecuta un *rollback* atómico que previene estados inconsistentes en las sucursales.

- ¿Los errores del dominio pueden explicarse con términos del negocio?

  *Sí.* Las excepciones y rechazos devuelven mensajes claros y contextualizados en el vocabulario de TecnoStock (por ejemplo, *"Producto reservado para [Cliente]"*, *"El apartado no cumple con el anticipo mínimo del 25%"*, o *"No aplica devolución por exceder el plazo de 30 días"*), facilitando la comprensión operativa para los cajeros y supervisores.